

EE8 — Count-to-5 Loop

C → Assembly → Machine Code

A program that counts from 0 to 5 in R2, then resets and repeats forever. Shown at three levels of abstraction. This is a basic test program that exercises LDI, ADD, LT, JZ, and JMP — no I/O required.

1 C (Equivalent Logic)

```

1  #include <stdint.h>
2
3  int main(void)
4  {
5      const uint8_t limit = 5;
6      const uint8_t step  = 1;
7
8      while (1) {
9          uint8_t counter = 0;
10
11         while (counter < limit)
12             counter += step;
13         /* counter == 5 here, then resets */
14     }
15 }
```

Note

The EE8 has no stack, no function calls, and 8-bit unsigned data. `uint8_t` variables map directly to registers; the outer `while(1)` mirrors the unconditional JMP back to the comparison.

2 EE8 Assembly

Register allocation: R0 = limit (5), R1 = step (1), R2 = counter, R3 = comparison result.

```

1  ;          Count-to-5 loop
2  ; R2 counts 0, 1, 2, 3, 4, 5 then resets to 0.
3  ;
4  0:      LDI  R0, 5          ; R0 = limit
5  1:      LDI  R1, 1          ; R1 = increment
6  2:      LDI  R2, 0          ; R2 = counter = 0
7  3:      LT   R2, R0         ; R3 = (R2 < R0) ? 1 : 0
8  4:      JZ   R3, 2          ; if R2 >= limit, restart
9  5:      ADD  R1, R2, R2     ; R2 = R2 + 1
10 6:      JMP  3              ; next iteration
```

3 EE8 Machine Code

Addr	Hex	Binary	Instruction
0	A005	1010 0000 0000 0101	LDI R0, 5
1	A401	1010 0100 0000 0001	LDI R1, 1
2	A800	1010 1000 0000 0000	LDI R2, 0
3	8800	1000 1000 0000 0000	LT R2, R0
4	CC02	1100 1100 0000 0010	JZ R3, 2
5	0680	0000 0110 1000 0000	ADD R1, R2, R2
6	B003	1011 0000 0000 0011	JMP 3
7-F	0000	0000 0000 0000 0000	(unused)

Note

Encoding notes: I-type (LDI) places Rd at bits [11:10]. R-type uses Rs1 [11:10], Rs2 [9:8], Rd [7:6]. J-type uses Rcond [11:10], address [3:0].

LT encoding: LT R2, R0 has Rs1=R2 at [11:10]=10, Rs2=R0 at [9:8]=00. The result always goes to R3 (hardwired by `flag_write`).

4 Execution Trace

Cycle	PC	R0	R1	R2	R3	Action
1	0	05	--	--	--	LDI R0, 5
2	1	05	01	--	--	LDI R1, 1
3	2	05	01	00	--	LDI R2, 0
4	3	05	01	00	01	LT: $0 < 5 \rightarrow R3=1$
5	4	05	01	00	01	JZ not taken ($R3 \neq 0$)
6	5	05	01	01	01	ADD: $R2 \leftarrow 1$
7	6	05	01	01	01	JMP 3
8	3	05	01	01	01	LT: $1 < 5 \rightarrow R3=1$
9	4	05	01	01	01	JZ not taken
10	5	05	01	02	01	$R2 \leftarrow 2$
11	6	05	01	02	01	JMP 3
⋮ (R2 increments: 3, 4)						
20	3	05	01	05	00	LT: $5 < 5 \rightarrow R3=0$
21	4	05	01	05	00	JZ taken $\rightarrow$ addr 2
22	2	05	01	00	00	LDI R2, 0 (reset)

The loop runs 5 iterations per cycle. R2 counts $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, then the comparison $5 < 5$ fails ($R3=0$), JZ branches back to address 2, and R2 resets to 0.